

三、微调方式

微调模型的代码主要结构:

- a. 准备微调模型的数据集，并结合数据集的格式以及模型要求，编写对应的数据转换代码(Dataset、DataLoader)；
- b. 加载预训练模型: 加载已经在其它数据集上训练好的模型结构及参数；
- c. 基于不同的微调方式，对加载好的模型进行"修正"；
- d. 定义优化器、损失函数、学习率更新器等；
- e. 微调模型(遍历训练数据，获取损失值，反向传播更新参数)；
- f. 评估模型: 使用验证集对微调后的模型进行效果验证评估；
- g. 模型持久化保存输出：按照既定的格式将模型保存到磁盘或者其它存储位置；

GPU 估算:

硬件依赖

* 估算值

方法	精度	7B	13B	30B	70B	110B	8x7B	8x22B
Full	AMP	120GB	240GB	600GB	1200GB	2000GB	900GB	2400GB
Full	16	60GB	120GB	300GB	600GB	900GB	400GB	1200GB
Freeze	16	20GB	40GB	80GB	200GB	360GB	160GB	400GB
LoRA/GaLore/BAdam	16	16GB	32GB	64GB	160GB	240GB	120GB	320GB
QLoRA	8	10GB	20GB	40GB	80GB	140GB	60GB	160GB
QLoRA	4	6GB	12GB	24GB	48GB	72GB	30GB	96GB
QLoRA	2	4GB	8GB	16GB	24GB	48GB	18GB	48GB

模型微调显存使用量(以全量微调为例):

模型微调/训练过程中，主要使用到显存的是以下几个部分:

- 1. 框架使用显存: PyTorch 框架相关 code 加载到显存中所使用到的大小，和显卡类型、框架类型版本是直接关联的，这部分可以直接测试一下即可；
- 2. 静态显存: 模型参数消耗的显存，梯度消耗的显存以及优化器消耗的显存；

假定模型参数量为 α ，以全量模型参数微调为例子：

模型的参数消耗显存：所有模型参数均需要存储到显存中，使用 fp32 存储，所需要消耗的显存就是 4α ；

梯度消耗的显存：梯度和模型可训练参数的量是完全相同的，使用 fp32 存储，所需要消耗的显存也就是 $4a$ ；

优化器消耗的显存：以 Adam 为例，使用 fp32 进行训练，Adam 中会存储 averaged momentum 和 variance 两部分，这两部分都和模型可训练参数量一样，所以显存消耗为： $2 \times 4a$ ；

总显存消耗量为： $4a + 4a + 2 \times 4a = 16a$ ；以一个 4B 的模型为例，总显存大小为： $4B \times 16 = 64G$ ；（ $1B=10^9$ ）

3. 中间激活状态消耗的显存；

存储计算过程中的临时结果值(反向传播时候需要)，这部分的计算量和网络的层次、维度大小以及批次大小有关；针对这部分占用显存过多的问题，在现有的 LLM 模型训练过程中，主要采用以下集中方式来解决：

- a. 小批次 + 梯度累加
- b. 梯度检查技术

4. 临时缓冲区占用的显存以及其它零散的显存占用：这部分是无法控制分析的；

微调方式

1. 全量微调

也就是会更新模型的所有参数,所有参数的 `requires_grad` 均为 `True`；

2. 冻结参数微调

冻结部分网络结构的参数，对剩余部分的参数进行更新；一般情况下会冻结网络前面一部分的参数，也可以冻结任何你想冻结的参数层。

```
# 待冻结的参数名称列表，eg：前n-2层的所有参数、所有FNN的参数...
freeze_parameter_names = [...]
```

```
for name, param in net.named_parameters():
    if name in freeze_parameter_names:
        param.requires_grad = False
```

Python

3. LoRA 微调

参考论文：<https://arxiv.org/pdf/2106.09685>

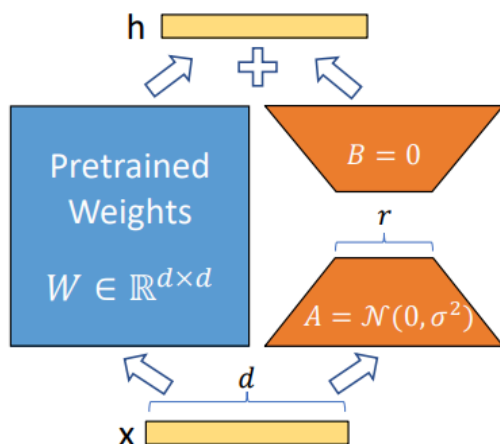


Figure 1: Our reparametrization. We only train A and B .

针对网络中的任何一个全连接子模块 $h=Wx$ 在参数微调的时候，都是计算 Loss 关于参数 W 的梯度，然后乘以学习率得到一个参数变化量 ΔW ，也就是一次反向传播的更新公式为: $W = W + \Delta W$ ，那么更新后的输出为: $h = (W + \Delta W)x = Wx + \Delta Wx$ ；从公式上可以看出我们可以将原始的 $h=Wx$ 看成 $h=Wx+Bx$ ，其中 W 固定冻结不进行更新，仅对 A 和 B 的参数进行反向传播更新，这个就是 LoRA 微调的过程，Lora 微调主要优点如下：

- 共享主干模型，不同任务使用不同的 LoRA 的模块，可以节省显存；
- 使用 LoRA 做参数高效微调，可以降低微调模型时候对显存的需求；
- 针对单任务的 LoRA 微调模型，在部署推理的时候，可以将 LoRA 模块参数和主干模型参数进行权重合并，这样相当于没有增加推理成本；
- LoRA 可以和其它高效微调方法一起结合使用；

NOTE:

初始化的时候，对矩阵 A 进行高斯随机初始化，对矩阵 B 进行全 0 初始化，这样可以保证在初始阶段仅只有主干分支生效，这样对于整个模型的微调相当于就是“参数没有更新”。

建议将参数 B 的学习率更改为参数 A 学习率的 16 倍。

建议 α 一般是 r 的两倍，参考：

https://blog.csdn.net/wwlsm_zql/article/details/135517355。

一般情况下，我们仅对 Attention 部分的参数进行 LoRA 微调，也就是 W_q 、 W_k 、 W_v 、 W_o ；

	# of Trainable Parameters = 18M						
Weight Type Rank r	W_q 8	W_k 8	W_v 8	W_o 8	W_q, W_k 4	W_q, W_v 4	W_q, W_k, W_v, W_o 2
WikiSQL ($\pm 0.5\%$)	70.4	70.0	73.0	73.2	71.4	73.7	73.7
MultiNLI ($\pm 0.1\%$)	91.0	90.8	91.0	91.3	91.3	91.3	91.7

其中矩阵 A、B 的维度大小 r 一般选择一个比较小的数即可；

	Weight Type	$r = 1$	$r = 2$	$r = 4$	$r = 8$	$r = 64$
WikiSQL($\pm 0.5\%$)	W_q	68.8	69.6	70.5	70.4	70.0
	W_q, W_v	73.4	73.3	73.7	73.8	73.5
	W_q, W_k, W_v, W_o	74.1	73.7	74.0	74.0	73.9
MultiNLI ($\pm 0.1\%$)	W_q	90.7	90.9	91.1	90.7	90.7
	W_q, W_v	91.3	91.4	91.3	91.6	91.4
	W_q, W_k, W_v, W_o	91.2	91.7	91.7	91.5	91.4

基于 peft 框架构建 LoRA 微调

Bash

```
# 安装库: https://github.com/huggingface/peft
pip install peft==0.12.0
```

Python

```
from transformers import AutoModelForSeq2SeqLM
from peft import get_peft_config, get_peft_model, LoraConfig, TaskType

model_name_or_path = "bigscience/mt0-large"
tokenizer_name_or_path = "bigscience/mt0-large"

peft_config = LoraConfig(
    task_type=TaskType.SEQ_2_SEQ_LM,
    inference_mode=False, r=8, lora_alpha=32,
    lora_dropout=0.1
)

# 迁移原始模型
model = AutoModelForSeq2SeqLM.from_pretrained(model_name_or_path)
# 模型结构的更改 -- 增加Lora模块
model = get_peft_model(model, peft_config)
model.print_trainable_parameters()

"trainable params: 2359296 || all params: 1231940608 || trainable%: 0.1915105310011
8282"
```

自定义 LoRA 模块：重写 linear 模块即可

Python

```

class LoraLinear(nn.Linear):

    def __init__(self, in_features: int, out_features: int):
        nn.Linear.__init__(self, in_features, out_features, **kwargs)
        self.r = 2 # 矩阵维度大小，也就是秩
        self.lora_alpha = 4 # 归一化参数大小，减小改变r时候需要重新训练的计算量
        self.scaling = self.lora_alpha / self.r
        self.lora_dropout = nn.Dropout(0.2) # dropout的系数
        self.lora_A = nn.Linear(in_features, self.r, bias=False)
        self.lora_B = nn.Linear(self.r, out_features, bias=False)

        self.in_features = in_features
        self.out_features = out_features

    def forward(self, x: torch.Tensor):
        # 这个就是执行的 torch.nn.Linear 的功能，对应的模型结构就是主干部分的模型结构；
        result = super().forward(x)

        x = x.to(self.lora_A.weight.dtype)

        # 这一部分是 LoRA 部分的模型结构；
        # 可以看出主干部分和 LoRA 部分的输入是相同的，都是 x
        lora_result = self.lora_B(self.lora_A(self.lora_dropout(x))) * self.scaling

        # 将主干部分的输出和 LoRA 部分的输出直接相加作为最终输出
        final_result = result + lora_result
        return final_result

```

4. QLoRA: Quantized LoRA

本身属于量化工作，通过定义一个 4NF 的精度单位将原模型的参数精度减小了数倍，从而大幅度的节省了训练时候的占用显存；

5. DoRA: Weight-Decomposed Low-Rank Adaptation

参考论文：<https://papers.cool/arxiv/2402.09353>

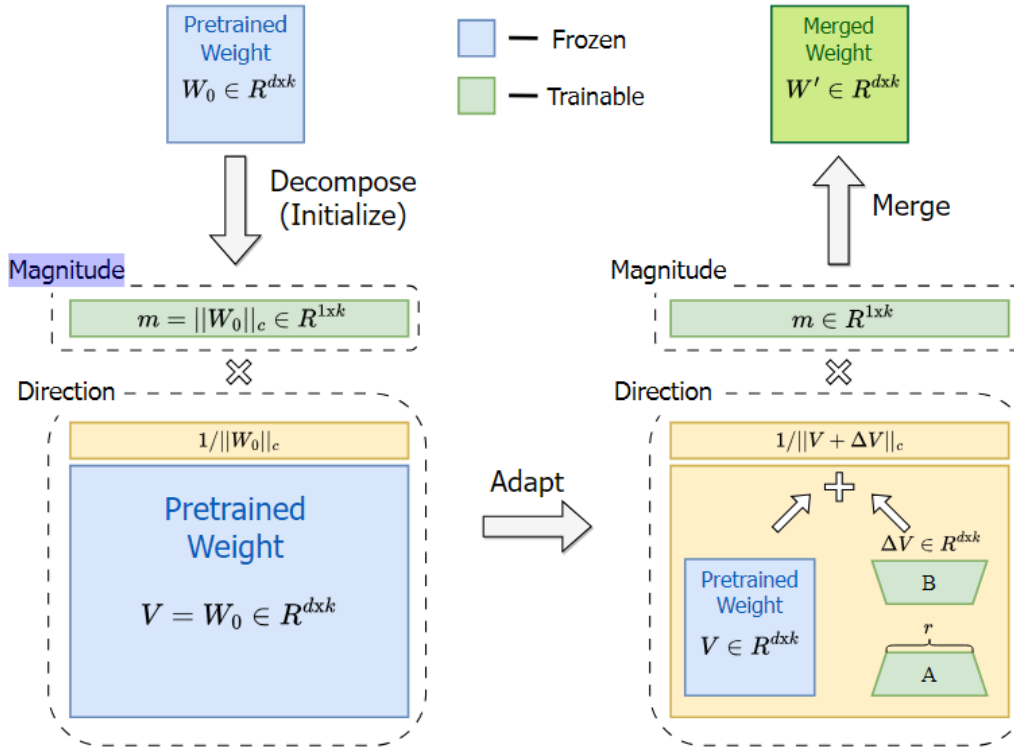


Figure 1. An overview of our proposed DoRA, which decomposes the pre-trained weight into *magnitude* and *direction* components for fine-tuning, especially with LoRA to efficiently update the direction component. Note that $|| \cdot ||_c$ denotes the vector-wise norm of a matrix across each column vector.

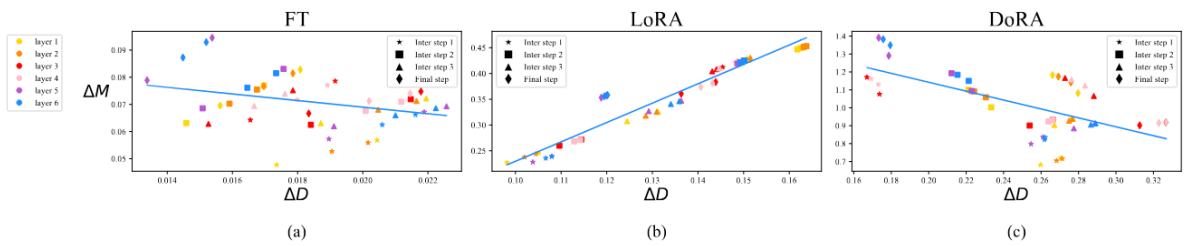


Figure 2. Magnitude and direction updates of (a) FT, (b) LoRA, and (c) DoRA of the query matrices across different layers and intermediate steps. Different markers represent matrices of different training steps and different colors represent the matrices of each layer.

一种通过拆分权重矩阵来提升训练效果的方法，可以增强 LoRA 的学习能力和训练稳定性；

将参数分为两个矩阵：幅度分量矩阵 M 和方向分量矩阵 V ，可以仅对 V 做 LoRA 微调，但是对 M 做全量微调，可以减少训练参数的数量。

全量微调可以实现仅对方向分量矩阵 V 做调整，而对幅度矩阵 M 不产生影响；但是 LoRA 会等比例的增加或者减少 M 和 V 的值；

6. rsLoRA: rank-stabilized LoRA

参考论文: <https://papers.cool/arxiv/2312.03732>

通过动态调整低秩矩阵的秩和缩放系数来进行适配，可以保证在训练过程中根据数据分布和模型需求进行优化，保证模型的稳定性，避免过拟合或者欠拟合。

7. PiSSA:

参考论文: <https://papers.cool/arxiv/2404.02948>

8. 其它微调方式:

▼ peft/utils/peft_types.py PeftType

Python

```
PROMPT_TUNING = "PROMPT_TUNING"
MULTITASK_PROMPT_TUNING = "MULTITASK_PROMPT_TUNING"
P_TUNING = "P_TUNING"
PREFIX_TUNING = "PREFIX_TUNING"
LORA = "LORA"
ADALORA = "ADALORA"
BOFT = "BOFT"
ADAPTION_PROMPT = "ADAPTION_PROMPT"
IA3 = "IA3"
LOHA = "LOHA"
LOKR = "LOKR"
OFT = "OFT"
POLY = "POLY"
LN_TUNING = "LN_TUNING"
VERA = "VERA"
```

参考代码:

Python

```
# -*- coding: utf-8 -*-
import numpy as np
import torch

def simu_full_training():
    from transformers import Qwen2ForCausalLM
    model_id = r"D:\huggingface\modelscope\hub\qwen\Qwen1___5-0___5B-Chat"

    model = Qwen2ForCausalLM.from_pretrained(model_id, device_map='cpu')

    cnt1 = 0
    cnt2 = 0
    for param in model.parameters():
        if param.requires_grad:
            cnt1 += 1
            cnt2 += torch.numel(param)
```

```

print(f"总待训练的参数数量<Tensor数量>为:{cnt1}")
print(f"总待训练的参数数量<Float浮点数>为:{cnt2} -- {cnt2 / np.power(10, 9)}")

def simu_freeze_training():
    from transformers import Qwen2ForCausalLM
    model_id = r"D:\huggingface\modelscope\hub\qwen\Qwen1___5-0___5B-Chat"

    model = Qwen2ForCausalLM.from_pretrained(model_id, device_map='cpu')
    # 参数冻结
    # freeze_layers = ['model.embed_tokens.weight']
    # for i in range(22):
    #     freeze_layers.append(f'model.layers.{i}.')
    # for name, param in model.named_parameters():
    #     if any([name.startswith(freeze_layer) for freeze_layer in freeze_layers]):
    #         param.requires_grad = False

    freeze_layers = [
        'model.embed_tokens.weight',
        '.self_attn.k_proj.', '.self_attn.v_proj.', '.self_attn.o_proj.',
        '.mlp.', 'norm'
    ]
    for name, param in model.named_parameters():
        for freeze_layer in freeze_layers:
            if freeze_layer in name:
                param.requires_grad = False
                break

    print("当前支持更新的参数列表为:")
    cnt1 = 0
    cnt2 = 0
    for name, param in model.named_parameters():
        if param.requires_grad:
            print(name)
            cnt1 += 1
            cnt2 += torch.numel(param)

    print(f"总待训练的参数数量<Tensor数量>为:{cnt1}")
    print(f"总待训练的参数数量<Float浮点数>为:{cnt2} -- {cnt2 / np.power(10, 9)}")

def simu_lora_training():
    from peft import get_peft_model, LoraConfig, TaskType
    from transformers import Qwen2ForCausalLM
    model_id = r"D:\huggingface\modelscope\hub\qwen\Qwen1___5-0___5B-Chat"

    model = Qwen2ForCausalLM.from_pretrained(model_id, device_map='cpu')

    peft_config = LoraConfig(
        task_type=TaskType.CAUSAL_LM,
        r=4, # 矩阵A和矩阵B的维度大小

```



```

        # target_modules=None, # 给定具体哪些模块进行Lora参数更新, peft(0.12.0版本)里None
        表示仅对query和value进行微调
        # target_modules=['k_proj', 'v_proj', 'q_proj'],
        target_modules='.*self_attn.*(k_proj|v_proj|value|o_proj)$',
        lora_alpha=8, # 用对控制lora模块输出值的缩放系数 scale = lora_alpha/r
        lora_dropout=0.0, # 针对lora模块输入数据的dropout系数
        inference_mode=False,
    )
    peft_model = get_peft_model(model, peft_config)

    print("=" * 50, "Lora模型结构")
    print(f"当前模型的参数数量:{len(list(model.parameters()))}")
    peft_model.print_trainable_parameters()
    print(peft_model)
    print("=" * 50)
    print("=" * 50, "Lora模型中支持梯度更新的参数:")
    for name, param in peft_model.named_parameters():
        if param.requires_grad:
            print(name)
    print("=" * 50)

    # 训练完成后, 可以将参数进行合并
    print(f"合并前参数数量:{len(list(peft_model.parameters()))}")
    merged_model = peft_model.merge_and_unload() # 参数合并
    print(f"合并后参数数量:{len(list(merged_model.parameters()))}")
    print(merged_model)

def tt_prefix_tuning():
    from transformers import T5Tokenizer, T5ForConditionalGeneration
    from peft import get_peft_model, TaskType, PrefixTuningConfig

    model_id = r"D:\cache\huggingface\hub\models--google-t5--t5-base\snapshots\a9723
    ea7f1b39c1eae772870f3b547bf6ef7e6c1"

    model = T5ForConditionalGeneration.from_pretrained(model_id, num_labels=13)
    print("=" * 50, "原始bert模型")
    print(f"当前模型的参数数量:{len(list(model.parameters()))}")
    print(model)
    print("=" * 50)

    config = PrefixTuningConfig(
        num_virtual_tokens=1000,
        token_dim=768,
        prefix_projection=True,
        encoder_hidden_size=768,
        inference_mode=False,
        task_type=TaskType.SEQ_2_SEQ_LM,
    )
    peft_model = get_peft_model(model, config)
    print("=" * 50, "Lora模型结构")
    print(f"当前模型的参数数量:{len(list(model.parameters()))}")

```

```

peft_model.print_trainable_parameters()
print(peft_model)
print("=" * 50)
print("=" * 50, "Lora模型中支持梯度更新的参数:")
for name, param in peft_model.named_parameters():
    if 'classifier' in name:
        param.requires_grad = True
    if param.requires_grad:
        print(name)
print("=" * 50)

if __name__ == '__main__':
    simu_lora_training()

```